Here, host2 pings the other two hosts—host1 and host3. Whenever either of the hosts makes a ping request, it is forwarded to the switch. This is indicated by arrows shown in blue. As this is the first request, the switch's flow table will not contain any entry. This is known as a *table miss*. Thus, the request will be forwarded to the controller.

The controller will instruct the switch with respect to the routing decision to be made and to also modify the flow table in the switch. This is shown by the arrows in green. The request will then be forwarded by the switch to the appropriate destination, i.e., to host1 and host3.

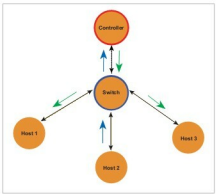

Figure 3: Network topology

The next time the same request is forwarded to the switch, the switch's flow table will contain an entry for that request and, thus, the switch itself will make routing decisions based on that entry, without the controller in action.

The explanation for the code to simulate the above topology is given below. Only the required extracts of the code are given. The accompanying lines of code demonstrate the extra header documents required for re-enacting wired systems.

```
#include<csma-module.h>
#include<ns3/ofswitch13-module.h>
```

The following lines of code create an object called 'hosts' of class *NodeContainer* common to all the other nodes. Here, three hosts are created.

```
NodeContainer hosts;
hosts.Create (3);

Ptr<Node> switchNode = CreateObject<Node> ();
//to create node for switch

CsmaHelper csmaHelper;
NetDeviceContainer hostDevices;
NetDeviceContainer switchPorts;
for(size_t i = 0;i<hosts.GetN();i++) //linking between
                                     hosts and switch
{
 NodeContainer pair (hosts.Get (i), switchNode);
      NetDeviceContainer link = csmaHelper.Install (pair);
      hostDevices.Add (link.Get (0));//two way linking
 switchPorts.Add (link.Get (1));
}


Ptr<Node> controllerNode = CreateObject<Node> ();
//to create node for
 controller
```

Here, ofswitch13 domain comes into action.

```
Ptr<OFSwitch13InternalHelper> of13Helper=Cr
eateObject<OFSwitch13InternalHelper>();
of13Helper->InstallController
(controllerNode);
//to install
        controller on node
of13Helper->InstallSwitch (switchNode,
switchPorts);
//to install OFSwitch
of13Helper->CreateOpenFlowChannels ();
```

```
//for creating channels            between Switch and
      controller

Ipv4AddressHelper ipv4helpr;      //set IPv4 addresses
Ipv4InterfaceContainer hostIpIfaces;
ipv4helpr.SetBase ("10.97.7.0", "255.255.255.0");
//IPv4 range starts from
10.97.7.0
hostIpIfaces = ipv4helpr.Assign (hostDevices);
```

Below lines will configure ping applications between hosts-

```
V4PingHelper pingHelper = V4PingHelper (hostIpIfaces.
GetAddress (1));
pingHelper.SetAttribute ("Verbose", BooleanValue (true));
ApplicationContainer pingApps1= pingHelper.Install (hosts.
Get (0));
pingApps1.Start (Seconds (1));
ApplicationContainer pingApps2 = pingHelper.Install (hosts.
Get (0));
pingApps2.Start (Seconds (1));
```

Here, two hosts and one object of class *ApplicationContainer* called *pingApps* is created.

Now the code for simulator to work-

```
Simulator::Stop (Seconds (10));   //simulation time is 10
                                   seconds
Simulator::Run ();
Simulator::Destroy ();
```

It is recommended that you save *ofswitch13-modify.cc* at this path (*ns-dev/scratch/*).

To run the program, use the following command:

```
$ ./waf --run ofswitch13-modify
```

The output is given in Figure 4.

As shown in the figure, the host with the IP address *10.97.7.2* pings the other two hosts, and the ping is